

UT-03: LECTURA Y ESCRITURA DE LA INFORMACIÓN. STREAMS.

¿Qué vamos a ver?

- Lectura y escritura de la información
- Flujos (streams)
- Tipos de flujos. Flujos de bytes y de caracteres.
- Clases relativas a flujos. Jerarquías de clases
- Utilización de flujos
 - Entrada/salida estándar
 - Entrada desde teclado

1. LECTURA Y ESCRITURA DE INFORMACIÓN

1.1. Salida de datos

La salida de datos por pantalla es una de las funcionalidades fundamentales al momento de desarrollar programas para usuarios, ya que, como programadores, probablemente necesitemos que nuestro programa se comunique con el usuario y le informe de todo lo que está sucediendo. Además, la salida de datos nos permite solicitar información al usuario.

1.2. Entrada de datos

Por otro lado, la entrada de datos también será una funcionalidad de gran utilidad, puesto que gracias a ella conseguiremos recolectar los datos con los que va a trabajar nuestro programa, solicitándolos al usuario, un soporte de almacenamiento u otros sistemas.

A todas las operaciones que constituyen un flujo de información del programa al exterior se les conoce como Entrada/Salida (E/S). Durante el curso veremos dos tipos de E/S:

- **E/S estándar** que se realiza con el terminal del usuario
- **E/S a través de fichero**, en la que se trabaja con fichero de disco.

En esta unidad de trabajo nos centraremos en la primera de ellas, y la segunda la veremos más adelante.

2. FLUJOS DE DATOS. STREAMS

Los flujos de datos son flujos de información entre el programa y el origen o destino de la información. Estos flujos son tratados en Java con objetos denominados streams.

Un stream (flujo en inglés) es un intermediario entre el programa y el origen o destino de la información.

Al utilizar los streams llevaremos a cabo los siguientes pasos:

- Lectura de la información
 1. Abrir un flujo de entrada
 2. Leer la información que contiene
 3. Cerrar el flujo de entrada
- Escritura de información
 1. Abrir un flujo de salida
 2. Escribir la información en el flujo de salida
 3. Cerrar el flujo de salida

3. TIPOS DE FLUJOS. FLUJOS DE BYTES Y DE CARACTERES

A la hora de utilizar los flujos podremos optar por trabajar con flujos de bytes o con flujos de caracteres.

3.1. Flujos de bytes

Los flujos de bytes nos permitirán tanto leer como escribir información en formato de bytes, sin necesidad de traducir dicha información. Estos nos serán de gran utilidad a la hora de leer o escribir información referente a texto, vídeo, audio, imágenes, etc...

3.2. Flujos de caracteres

A diferencia de los flujos de bytes, estos únicamente están enfocados a la entrada y salida de datos de tipo texto, aunque en realidad siguen siendo secuencias de bytes.

4. CLASES RELATIVAS A FLUJOS. JERARQUÍAS DE CLASES

El paquete `java.io` contiene todas las clases relacionadas con las funciones de entrada (input) y salida (output).

Las clases de este paquete permiten deducir su función a partir de las palabras que componen sus nombres. Véase la siguiente tabla:

Palabra	Significado
InputStream/OutputStream	Lectura/Escritura de byte
Reader/Writer	Lectura/Escritura de caracteres
File	Archivos
String, CharArray, ByteArray, StringBuffer	Memoria
Piped	Tubo de datos
Buffered	Buffer
Filter	Filtro
Data	Intercambio de datos en formato Java
Object	Persistencia de objetos
Print	Imprimir

4.1. Jerarquías de clases Entrada/Salida de bytes

4.1.1. Entrada/salida con `InputStream` y `OutputStream`

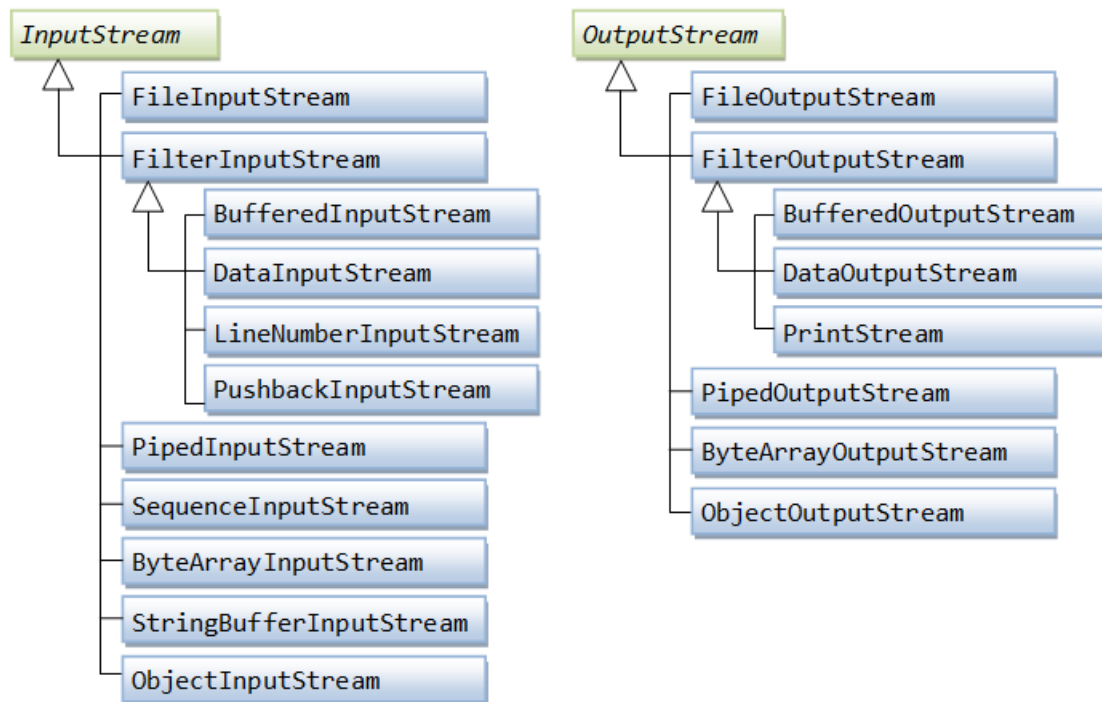
`java.lang.System` es una clase que posee multitud de pequeñas clases relacionadas con la configuración del sistema. Entre ellas están:

- **.in** que es un `InputStream` que representa la entrada estándar (normalmente el teclado). Puede utilizar los métodos:

- **read()**: devuelve el código correspondiente al carácter que se ha introducido por el teclado leyendo del buffer de entrada o “-1” si no se ha introducido ningún carácter.
- **skip(n)**: ignora los n caracteres siguientes a la entrada.
- **.out** que es un OutputStream que representa a la salida estándar (normalmente la pantalla). Puede utilizar los métodos:
 - **print(a)**: imprime a en la salida, donde a puede ser cualquier tipo primitivo de Java, ya que Java hace su conversión automática a cadena.
 - **println(a)**: es idéntico a print(a) salvo que con println() se imprime un salto de línea al final de la impresión de a.
- **.err** que es un OutputStream que representa a la salida en caso de error.

```
public static void main(String[] args) {  
    try{  
        System.out.print("Dame un carácter");  
        int character = System.in.read();  
        System.out.println((char)character);  
    } catch (Exception ex){  
  
    }  
}
```

A continuación, tenemos un esquema en el que podemos ver las distintas clases con las que se puede trabajar con entrada y salida de bytes.



4.2. Jerarquías de clases Entrada/Salida de caracteres

4.2.1. Entrada/salida con `InputStreamReader` y `OutputStreamWriter`

El hecho de que las clases `InputStream` y `OutputStream` usen el tipo `byte` para la lectura, complica mucho su uso.

Las soluciones que se dieron fueron las clases `InputStreamReader` y `OutputStreamWriter`.

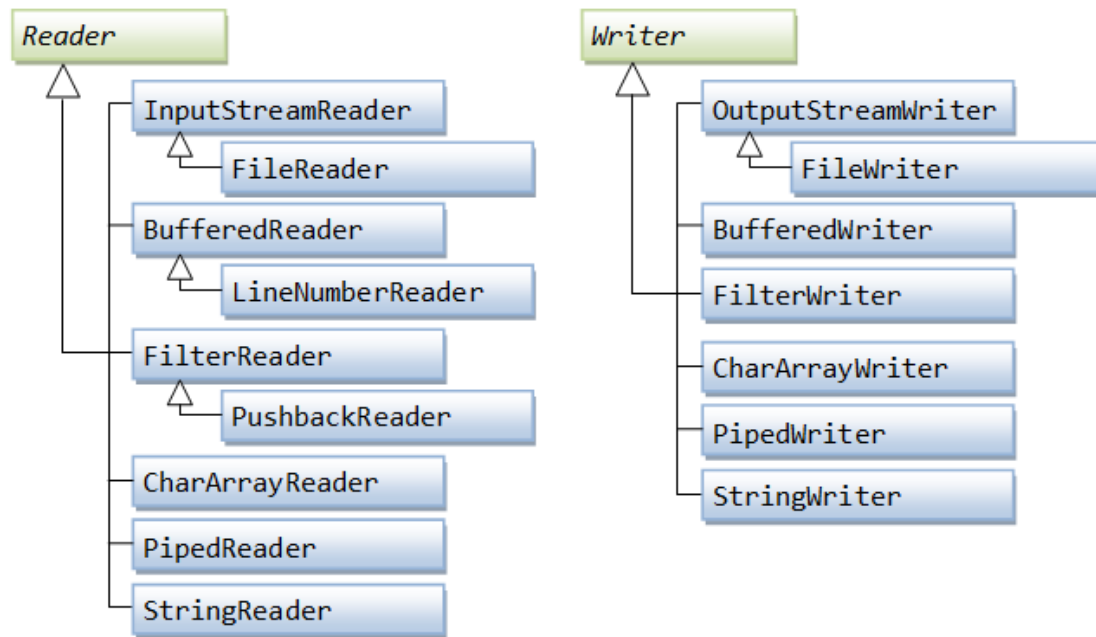
Estas clases se utilizan para convertir secuencias de bytes en secuencias de caracteres y permiten construir objetos de este tipo a partir de objetos `InputStream` u `OutputStream`.

El constructor de la clase `InputStreamReader` requiere un objeto `InputStream`.

```
InputStreamReader stdin=new InputStreamReader(System.in);
```

Estas clases trabajan con el formato de carácter UNICODE, de 16 bits.

A continuación, tenemos un esquema en el que podemos ver las distintas clases con las que se puede trabajar con entrada y salida de caracteres.



Ejemplo de lectura usando InputStreamReader

```

//Importamos la librería de InputStreamReader para poder utilizarla
import java.io.InputStreamReader;

public class EntradaSalida {

    public static void main(String[] args) {
        try{
            //Creamos el lector escuchando de System.in
            InputStreamReader stdin = new InputStreamReader(System.in);
            //Creamos una variable capaz de almacenar hasta 1024
            caracteres
            char mensaje[] = new char[1024];
            int tamMensaje;

            //Solicitamos al usuario un mensaje
            System.out.print("Dame un mensaje: ");

            //Leemos el mensaje y almacenamos su tamaño
            tamMensaje = stdin.read(mensaje);

            //Escribimos el mensaje si la lectura no ha devuelto un error
            System.out.println((tamMensaje > -1)? mensaje :
            "");

        } catch (Exception ex){

```

```
    }  
  }  
}
```

4.2.2. Entrada/salida con clases buffered

La palabra buffered hace referencia a la capacidad de almacenamiento temporal en la lectura y la escritura.

Los datos se almacenan en una memoria temporal antes de ser realmente leídos o escrito.

Se trata de cuatro clases:

- **BufferedInputStream**
- **BufferedOutputStream**
- **BufferedReader**
- **BufferedWriter**

Trabajan con métodos distintos, pero suelen utilizar los mismos flujos de entrada que podrán ser bytes (Input/OutputStream) o caracteres (Reader/Writer).

La clase `BufferedReader` aporta el método `readLine()` que permite leer caracteres hasta la presencia de null o del salto de línea y los devuelve como un String sin incluir '\n';

Este método puede lanzar una excepción `java.io.Exception`.

Esta clase usa un constructor que acepta objetos `Reader` (y por lo tanto `InputStreamReader`, ya que descende de esta).

```
BufferedReader in=new BufferedReader(newInputStreamReader(System.in));  
String frase=in.readLine();
```


Ejemplo de lectura escritura con BufferedReader:

```
//Importamos la librerías BufferedReader e InputStreamReader para poder
utilizarlas
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class EntradaSalida {

    public static void main(String[] args) {

        try{
            String texto="";

            //Obtención del objeto lector
            InputStreamReader stdin = new InputStreamReader(System.in);

            //Obtención del BufferedReader
            BufferedReader bufferReader = new BufferedReader(stdin);

            //Solitamos un texto al usuario
            System.out.print("Dame un texto: ");
            texto = bufferReader.readLine();

            //Escribe la línea de texto por pantalla
            System.out.println(texto);

        }catch (Exception ex){

        }

    }
}
```

4.2.3. Entrada con clase Scanner

La clase Scanner está disponible desde Java 5, en el paquete java.util.

<https://docs.oracle.com/javase/8/docs/api/java/util/Scanner.html>

Dispone de métodos para leer valores de entrada de diferentes tipos de datos (nextInt(), nextDouble(), nextLine(),...)

Para utilizar la clase tenemos que instanciarla indicando el medio de entrada de información. Si es la entrada estándar se indicará System.in o si es desde un fichero en disco, la ruta de directorios del fichero.

```
Scanner sc = new Scanner(System.in);
```

Con la instancia sc creada, podemos invocar sus métodos.

```
sc.nextLine(); Lee una línea de texto de teclado;
```

A continuación tenemos un cuadro con los distintos métodos que podemos utilizar con esta clase:

Método	Significado	Ejemplo
nextByte()	Lectura de un dato byte	byte b = sc.nextByte();
nextDouble()	Lectura de número real	double d = sc.nextDouble();
nextFloat()	Lectura de número real	float f = sc.nextFloat();
nextInt()	Lectura de número entero	int i = sc.nextInt();
nextLong()	Lectura de entero largo	long l = sc.nextLong();
nextShort()	Lectura de entero corto	short s = sc.nextShort();
next()	Lectura de String	String p = sc.next();
nextLine()	Devuelve la línea entera como un String. Lee el salto de línea \n	String o = sc.nextLine();

4.2.3.1. Funcionamiento de la clase Scanner

Cuando se introducen caracteres por teclado, el objeto Scanner toma toda la cadena introducida y la divide en elementos llamados tokens.

Los tokens resultantes pueden convertirse en valores de diferentes tipos utilizando los métodos apropiados para cada tipo.

El carácter predeterminado que sirve de separador de tokens es el espacio en blanco. Para usar otro carácter diferente tendremos que indicarlo a la hora de instanciar la clase Scanner usando `useDelimiter("carácter");`

Por ejemplo, si introducimos la cadena: 12 20.001 Lucas w

Los tokens que se crean son:

- 12
- 20.001
- Lucas
- W

Cuando en un programa se leen por teclado datos numéricos y datos de tipo carácter o String debemos tener en cuenta que al introducir los datos y pulsar intro estamos también introduciendo en el buffer de entrada el intro.

Este carácter no es leído por `nextInt`, `nextDouble`,... por lo que permanecerá en el buffer y podría ser leído por `nextLine()` en una siguiente instrucción.

La solución es poner una línea `sc.nextLine();` que limpie el buffer.

4.3. Conversión a otros tipos de datos

En algunas ocasiones es conveniente leer datos por teclado como una cadena de caracteres String y realizar la conversión al tipo de dato adecuado. Para ello Java dispone de métodos en las siguientes clases:

- Clase Integer de la librería `java.lang`
 - `public static int parseInt(String s);` //recibe String y retorna entero
 - Ej. `int num = Integer.parseInt(sc.nextLine());`

- Clase Float de la librería java.lang
 - `public static float parseFloat(String s); //recibe String y retorna float.`
 - Ej. `float numf = Float.parseFloat(sc.nextLine());`
- Clase Double de la librería java.lang
 - `public static long parseDouble(String s); //recibe String y retorna double`
 - Ej. `double numReal = Double.parseDouble(sc.nextLine());`
- Clase Boolean de la librería java.lang
 - `public static boolean parseBoolean(String s); //recibe String y retorna boolean`
 - Ej. `boolean boleano = Boolean.parseBoolean(sc.nextLine());`

5. SALIDA DE DATOS ESTÁNDAR (POR CONSOLA)

Como ya hemos comentado anteriormente, para devolver una cadena de caracteres al flujo de datos estándar, es decir, que salga por consola debemos utilizar la clase `System.out` y sus funciones `println()` o `print()`, según necesitemos escribir con un salto de línea tras la cadena de caracteres o no.

Estas funciones reciben como parámetro un `String`, pero nos está permitido introducir un valor de un tipo de datos primitivo y Java se encargará de convertirlo a `String` mediante una conversión implícita. Por ejemplo:

```
int numeroEntero = 567;
```

```
System.out.print(numeroEntero);
```

Se realizará una conversión implícita, de tal forma que `numeroEntero` se convertirá a un `String` y la consola nos devolverá `"567"`.

Además de la conversión implícita de datos primitivos a `String`, vamos a poder utilizar el operador `+` para encadenar, o concatenar, cadenas de caracteres, pudiendo construir mensajes a través de la unión de varios `Strings`, ya sean variables, constantes o literales. Por ejemplo:

```
final String inicioTexto = "Tengo ";  
  
int numAnyos = 21;  
  
System.out.println(inicioTexto + numAnyos + " años");
```

Por consola devolverá: "Tengo 21 años";

5.2. Aplicar formato a la salida estándar

En ocasiones desearemos aplicar un formato a la salida por consola. Esto suele ocurrir principalmente con salida de numéricas.

Para esto haremos uso de la función "format" de la clases System.out.

```
System.out.format(String formato, Object... args)
```

formato: es una cadena que especifica el formato que debe utilizarse.

args: es una lista de las variables que desea imprimir utilizando ese formato. Por ejemplo:

```
System.out.format("El valor de la variable float es %f, mientras que el  
valor de la variable entera es %d y la cadena es %s", VarReal, VarEntera,  
VarCadena);
```

La cadena de formato contiene unos caracteres especiales que dan formato a los argumentos de Object... args. Son **especificadores de formato** que comienzan con un signo de porcentaje (%) y un carácter que indica el tipo de argumento para ser formateado.

- **%d:** formatea el valor entero (int) como un valor decimal.
- **%f:** da formato a un valor en coma flotante(float, double) como valor decimal.
- **%n:** da salida a un final de línea específica de la plataforma.
- **%x:** formatea un entero como un valor hexadecimal.
- **%s:** formatea cualquier valor como una cadena.

Además, puede presentar unos elementos adicionales de manera opcional. Estos elementos deben aparecer en el orden que se indica. Trabajando desde la derecha, los elementos opcionales son:

- **%**: Inicio de especificador de formato
- **argumentIndex\$**: indica el argumento que estamos utilizando mediante su posición en el listado de argumentos.
- **+**: muestra el signo positivo en número positivos.
- **0**: rellena con "0" a la izquierda hasta alcanzar la longitud que se le ha indicado.
- **numeroAncho**: indica la longitud de dígitos que debe tener.
- **.numeroDecimales**: indica el número de decimales que debe mostrarse a la derecha del punto.
- **especificadorFormato**: indica el formato final que debe darse (listado ya visto anteriormente)

```
long n = 461012;
System.out.format("%d%n", n); // --> "461012"
System.out.format("%08d%n", n); // --> "00461012"
System.out.format("%+8d%n", n); // --> " +461012"
double pi = Math.PI;
System.out.format("%f%n", pi); // --> "3.141593"
System.out.format("%.3f%n", pi); // --> "3.142"
System.out.format("%10.3f%n", pi); // --> " 3.142"
```

Ejemplos con varios argumentos:

```
int edad = 20;
String nombre = Lola;
System.out.format("La persona se llama %s y tiene %d años\n", nombre,
edad);
```

Resultado: "La persona se llama Lola y tiene 20 años"

```
int cantidad = 10;
String producto = "mesas";
double precio = 125.55;
double total = cantidad * precio;
System.out.format("Para una cantidad de %d %s el total de la compra es
%.2f \n", cantidad, producto, total);
```

Resultado: "Para una cantidad de 10 mesas el total de la compra es 1255,50"

5.3. Caracteres de barra invertida

Existen una serie de caracteres especiales denominados de barra invertida o literales especiales que se componen del símbolo de barra invertida “\” y otro carácter adicional al que este da un significado especial al escribir un String. A continuación, podemos observar de cuales se trata y para qué se utilizan.

Caracteres literales especiales

Códigos	Significado
'\n'	Nueva línea
'\r'	Retorno de carro
'\t'	Tabulación
'\"'	Comilla simple
'\"'	Comilla doble
'\\'	Barra inclinada inversa

Ejemplo:

```
System.out.print("Hola me llamo Javier, pero me puedes llamar  
\"Javi\"\\n");  
System.out.print("¿Cuál es tu nombre?\\n");
```

Resultado:

Hola me llamo Javier, pero me puedes llamar "Javi"

¿Cuál es tu nombre?